Recursion and Storage Classes

Reference Variable

1. C++ introduces a new kind of variable known as reference variable.
2. A reference variable provides an alias(alternate name) for a previously defined variable.
3. e.g. if we make a variable sum a reference to total, then sum and total can be used interchangeably to represent that variable.
4. A reference variable is created as follows:

data-type & variable-name = variable name;

Example:

float total = 100;

float &sum=total;

total is a float type variable that has already been declared, sum is alternative name declared to represents the variable total. Both the variables refer to the same data object in the memory.

cout<<total;

cout<<sum;

Call By reference

1. A function call passes arguments by value .
2. The called function creates a new set of variables and copies the value of arguments into them.
3. The function doesn't have access to the actual variables in the calling program and can only work on the copies of value.
4. This mechanism is fine if function does not need to alter the values of the original variables in the calling program .
5. But there may arise situation where we would like to change the values of variables in the calling program.
6. Reference variables in C++ permits us to pass parameters to the function by reference.
7. When we pass arguments by reference the formal arguments in the calling function become aliases to the actual arguments in the calling function.
8. This means that when the function is working with its own arguments it is actually working on the original data.

```
using namespace std;

void swap(int &a, int &b)
{
	int t = a;
	a = b;
	b = t;
	cout << "a = " << a << endl;
	cout << "b = " << b << endl;

}
int main()
{
	int a = 4, b = 5;
	swap(a, b);
	cout << "a = " << a << endl;
	cout << "b = " << b << endl;
	system("pause");
	return 0;
}
```

Return by reference

A function can also return a reference. Consider the following function:

```
#include<iostream>
using namespace std;

int & max(int &x, int &y)
{
     if (x > y)
          return x;
     else
          return y;
}
int main()
{
     int a = 4, b = 5;

     max(a, b) = -1;
     cout << a << endl;
     cout << b << endl;

     system("pause");
     return 0;
}
```

Inline Function

To eliminate the cost of calls to small functions C++ proposes a new feature called inline function. An inline function is a function that is expanded in line when it is invoked that is compiler replaces the function call with the corresponding function code. The inline functions are defined as follows:

```
inline  function header
{
Function-body
}
```

Example:

```
Inline double cube(double a)
{  return  a*a*a;
}
```

Remember that inline keyword merely send a request not a command, to compiler. The compiler may ignore this request if the function definition is too long or too complicated and compile the function as normal function.

Some of the situations where inline expansion may not work are:

1. For functions returning values , if a loop, a switch or a goto exists.
2. If function contains static variables.
3. If inline functions are recursive.

Recursive Functions

A recursive function is defined as a function that calls itself to solve a smaller version of its task until a final call is made which does not require a call to itself. Every recursive solution has two major cases, they are:

Base Case: program is simple enough to be solved directly without making any further calls to the same function.

Recursive Case:

1. The problem at hand is divided into simpler sub parts.
2. The function calls itself but with sub parts of the problem obtained in the first step.
3. result is obtained by combining the solutions of simpler sub parts.

To understand recursive functions, let us take an example of calculating factorial of a number. To calculate !n , we have to multiply the number with factorial of the number that is one less than the number. In other words:

!n = n*!n-1

Let us assume we need to find the value of !5

!5 = 5*4*3*2*1=120

This can be written as

!5=5*!4, where !4=4*3*2*1

Therefore, the series of problem and solution can be given as :

Problem	Solution
!5	5*4*3*2*!1
=5*!4	=5*4*3*2*1
=5*4*!3	=5*4*3*2
=5*4*4*!2	=5*4*6
=5*4*3*2*!1	=5*24
	=120

For the factorial function

Base Case: when n=1, because if n=1, the result will be 1 as !1=1

Recursive Case: factorial function will call itself but with a smaller value of n , this case be given as:

factorial(n) = n*factorial(n-1)

```cpp
#include<iostream>
using namespace std;
int fact(int);
int main()
{
    int num;
    cout << " enter the number"<<endl;
    cin >> num;
    cout << "Factorial of  " << num << " = " << fact(num) << endl;
        system("pause");
    return 0;
}
int fact(int n)
{
    if (n == 1)
        return 1;
    else
        return (n*fact(n - 1));
}
```

From this example , let us analyze the basic steps of a recursive program:

1. Specify the base case which will stop the function from making a call to itself.

2. Check to see whether the current value being processed matches with the value of base case. If yes, process the return value.

3. Divide the problem into smaller or simpler sub-problem.

4. Call the function from sub problem.

5. Combine the results of sub-problems.

6. Return the result of entire problem.

Types of Recursion

Any recursive function can be characterized based on the following:

1. Whether the function calls itself directly or indirectly.(direct or indirect recursion)

2. Whether any operation is pending at each recursive call.(tail – recursive or not)

3. The structure of the calling pattern .(linear or tree recursive)

Direct Recursion

A function is said to be directly recursive if it explicitly calls itself.

```
int  func(int n)
{
if(n==0)
return n;
else
return (func(n-1))
}
```

Here, the func() calls itself for all positive values of n, so it is said to be directly recursive.

Indirect recursion

A function is said to be indirectly recursive if it contains a call to another function which ultimately calls it. Following functions are indirectly recursive as they both call each other:

```
int func1(int n)
{
if(n==0)
Return n;
else
return  func2(n);
}
int func2(int x)
{ return func1(x-1);
}
```

Tail Recursion

1. A recursive function is said to be tail recursive if no operations are pending to be performed when the recursive function returns to its caller .

2. When the called function returns, the returned value is immediately returned from the calling function.

3. Tail recursive functions are highly desirable because they are much more efficient to use as the amount of information that has to be stored on the system stack is independent of the number of recursive calls.

```
#include<iostream>
using namespace std;
int fact(int);
int fact1(int, int);
int main()
{
    int num;
    cout << " enter the number"<<endl;
    cin >> num;
    cout << "Factorial of  " << num << " = " << fact(num) << endl;
        system("pause");
    return 0;
}
int fact(int n)
{
    return fact1(n, 1);
}
int fact1(int n , int res)
{
        if (n == 1)
        return res;
    else
        return fact1(n-1,n*res);
}
```

Converting Recursive functions to tail Recursive

1. A non tail recursive function can be converted into a tail-recursive function by using an auxiliary parameter as in case of factorial function.

2. The auxiliary parameter is used to form the result.

3. When we use such a parameter , the pending operation is incorporated into the auxiliary parameter so that recursive call is no longer has a pending operation.

4. We generally use auxiliary function while using auxiliary parameter.

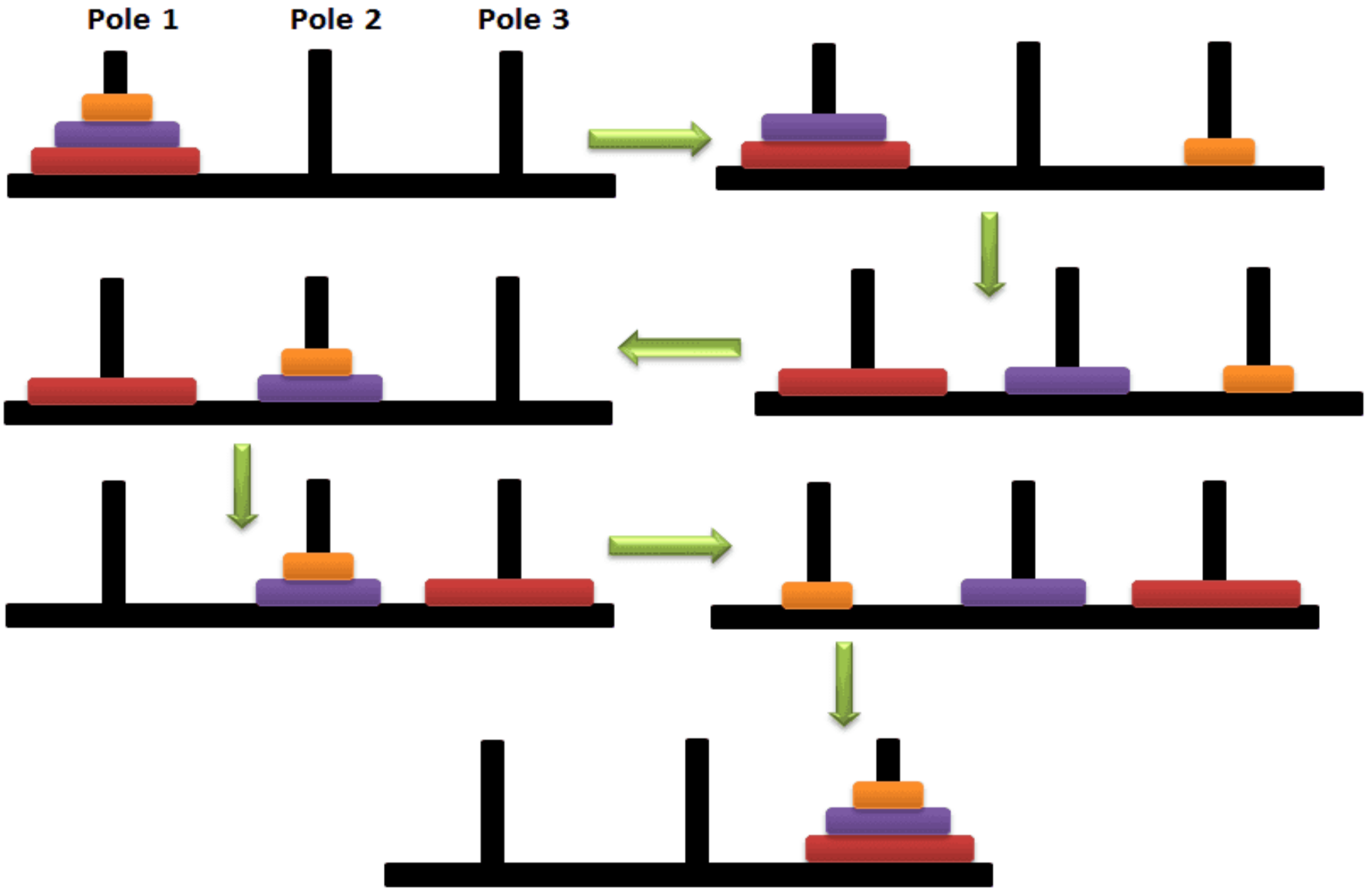
5. This is done to keep the syntax clean and to hide the fact that auxiliary parameters are needed.

Tower of Hanoi

Tower of Hanoi is also called as **Tower of Brahma** or **Lucas Tower**. It is one of the most popular problem which makes you understand the power of recursion. **Tower of Hanoi** is a mathematical puzzle which consist of **3 poles** and number of **discs** of different sizes. Initially all the discs will be places in the single pole with the largest disc at the bottom and smallest on the top. We need to move all the disc from the first pole to the third pole with the smallest disc at the top and the largest at the bottom under the below conditions

1. Only one disc can be moved at a time.

2. Larger disc cannot be placed on a smaller disc.


Pole 1
Pole 2
Pole 3

Tower of Hanoi algorithm

To summarize , the solution to our problem of moving n rings from A to C using B as spare can be given as:

Base case:

If n=1

Move the rings from A to C using B as spare.

Recursive case:

1. Move (n-1) rings from A to B using C as spare.
2. Move the one ring left on A to C using B as spare.
3. Move (n-1) rings from B to C using A as spare.

```cpp
#include<iostream>
using namespace std;
void move(int, char, char, char);
int main()
{
    int n;
    cout << "Enter the number of rings:" << endl;
    cin >> n;
    move(n, 'A', 'C', 'B');
    system("pause");
    return 0;
}
```

```cpp
void move(int n, char source, char dest, char spare)
{
    if (n == 1)
    {
        cout << "Move from " << source << " to " << dest<<endl;
    }
    else
    {
        move(n - 1, source, spare, dest);
        move(1, source, dest, spare);
        move(n - 1, spare, dest, source);

    }
}
```